

Lab 6: Hashing in Exhibition Layout Design

Group 1: State Deduplication

Abhigya Koirala

Problem Summary

When generating candidate exhibition layouts using algorithms like backtracking or greedy search, the system can create the same layout multiple times through different paths. Our subproblem involves designing an $O(1)$ expected-time deduplication system to check if a specific layout state has already been evaluated. This will help reduce the state space and avoid unnecessary calculations.

Data Representation

A **Layout** is essentially an unordered set of **Placement** objects. Each **Placement** is a complex data structure that consists of:

- **Location loc**: A nested structure with room id and wall id.
- **artwork_id**: An unsigned 32-bit integer.
- **orientation**: An unsigned 16-bit integer that indicates rotation in degrees.

Hash Function Design and Justification

To ensure high performance, I used Zobrist Hashing. Instead of sorting the vector of placements and converting it into a canonical string, which takes $O(N \log N)$ time and uses a lot of memory, the system caches a consistent, pseudo-random 64-bit integer for each unique **Placement** generated once using a `std::mt19937_64` generator and reused after.

The hash for an entire **Layout** is calculated by taking the XOR sum of its individual placement hashes:

$$H(Layout) = h(p_1) \oplus h(p_2) \oplus \dots \oplus h(p_n)$$

Since the XOR operation is both commutative and associative, the resulting hash does not depend on the order of the placements. This lowers the hashing time complexity to an efficient $O(N)$ bitwise calculation. An additional advantage is that the hash can be updated incrementally in $O(1)$ time when adding or removing a placement, rather than recomputing the entire layout hash from scratch.

Collision Handling Strategy

We handle hash collisions using a hybrid approach. The `std::unordered_set` manages bucket collisions through separate chaining. However, to address mathematical hash collisions, the slight chance that two different layouts yield the same 64-bit XOR sum, I kept the original `canonical_string()` method strictly for the `operator==` overload.

The quick Zobrist hash quickly finds the right bucket, and the slower canonical string matching is only triggered to confirm equality if a collision occurs, ensuring correctness (no false positives) while preserving fast average case performance. This distinguishes between bucket collisions handled internally by the hash table and true hash collisions, which are resolved through equality checking.

Small Dataset Demonstration

Tests on the `small_layouts.txt` dataset ($N = 4$) produced these results:

```
Layouts read: 4
Unique layouts: 2
Duplicates: 2
Slow Method (String Hashing) Time: 0.007125 ms
Fast Method (Zobrist Hashing) Time: 0.062125 ms
```

Analysis: In this small dataset, the Zobrist method is actually slower. This is because of the fixed overhead involved in setting up the `std::mt19937_64` RNG and filling the initial `unordered_map`. For just 4 items, sorting the vector is nearly instantaneous.

Large Dataset Performance Test

Testing on the `large_layouts.txt` dataset showed the clear advantage of the Zobrist hash:

```
Layouts read: 230
Unique layouts: 200
Duplicates: 30
Slow Method (String Hashing) Time: 0.94025 ms
Fast Method (Zobrist Hashing) Time: 0.409417 ms
Speedup: 2.29656x faster!
```

As the number of elements and layouts increases, the $O(N)$ XOR computation combined with $O(1)$ expected lookup avoids the $O(N \log N)$ sorting cost, leading to a 2.3x performance improvement.

Reflection

What happens when the load factor increases?

As the load factor ($\alpha = n/m$) of our `std::unordered_set` grows, the number of items stored increases in relation to the number of available buckets. This raises the chances of bucket collisions. While C++ manages this well with separate chaining, a high load factor can reduce our expected $O(1)$ lookup time to $O(N)$ in heavily populated buckets. Since we use a strong 64-bit Zobrist hash, any non-bucket hash collisions remain statistically insignificant.

What failure cases did you observe?

One notable failure case is related to spatial symmetry. If the gallery allows a layout to be rotated 180 degrees or mirrored across a room, the current hashing method treats these as completely unique states. To effectively eliminate symmetrical duplicates, the system would need a `normalize()` step to align all symmetrical layouts into one canonical orientation before applying the Zobrist hash.